Learning to Play Battleship with Tabular Q-Learning over a Local Cell Abstraction

Priya Raghunathan[1], **Mateo Villalobos Ríos**[2], **Ingrid Solheim**[1]

[1]Department of Computer Science, Fjellheim University of Technology, Trondheim, Norway
[2]Facultad de Ingeniería, Universidad Austral de Cerro Alto, Valparaíso, Chile
p.raghunathan@fjellheim-tech.no, mvillalobos@ing.uacerroalto.cl, ingrid.solheim@fjellheim-tech.no

Abstract

Battleship is a two-player game of imperfect information in which the opponent is passive: each player independently searches a hidden board, and the winner is whoever needs fewer shots. Its literal state — a 10×10 tracking grid with four cell states — has 4^{100} configurations, so the game is normally attacked either with hand-written hunt/target heuristics or with deep function approximation. We take the third route: instead of replacing the table with a network, we shrink the problem until a table is the right tool. Each candidate cell is described by what its four orthogonal directions run into first, plus two global features, and the description is canonicalised under the dihedral symmetry of the board. Plain tabular Q-Learning over this action-attached abstraction fills $2{,}839$ entries in 55 seconds of single-core CPU time and clears a board in 52.1 shots, against 56.8 for a tuned hunt/target baseline with a parity sweep and 94.9 for random play; head to head it wins 181 of 300 games against the heuristic. An ablation over every design decision shows that the binding constraint is not the size of the table but the granularity of the key: *deleting* a feature that appears informative — the size of the smallest surviving enemy ship — yields a 919-entry table that clears boards in 48.2 shots and beats the heuristic 205–95. Because the policy is a table over local descriptions, it can be read out entry by entry as a list of rules.

Introduction

Battleship is played by two people who cannot see each other's board. Each places a fleet of five ships on a 10×10 grid and then the players alternate calling single shots, learning only whether each shot hit, missed, or sank a ship. What makes the game unusual among two-player games of imperfect information is that neither player can do anything to the other: the boards are fixed at setup, and nothing a player does changes what the opponent will find. Each side is really solving a private search problem — *find seventeen occupied cells among one hundred* — and the winner is simply whoever finishes first. Battleship therefore isolates the search half of imperfect-information play from the game-theoretic half that dominates poker (Zinkevich et al. 2007; Brown and Sandholm 2018), and it comes with a metric that needs no opponent at all: how many shots does a policy need to clear one board?

That clean measurement has made Battleship a recurring testbed. It is one of the three headline domains of POMCP (Silver and Veness 2010) and reappears as a benchmark for learned belief-state inference (Igl et al. 2018), both times described as a POMDP with roughly 10^{18} states. Cognitive scientists use it to study goal-directed information gathering, from question asking as program generation (Rothe, Lake, and Gureckis 2017, 2018) to language-informed program sampling (Grand et al. 2024) and rational exploration by language-model agents (Grand et al. 2026), which keeps it an active domain as of 2026.

The size of the state space is the reason the game is normally attacked from one of two extremes. At one end sit hand-written rules: hunt until something is hit, then target around the wound, and sweep a parity lattice while hunting (Berry 2011). At the other end sit function approximators, from neural Battleship players (Clementis 2013, 2014) to online Monte-Carlo planners (Silver and Veness 2010) and deep variational belief models (Igl et al. 2018). The literal tracking grid has 4^{100} configurations, so a table over states is not merely large, it is pointless: two games would never share a state, and the tabular convergence guarantee (Watkins and Dayan 1992), which explicitly requires the action values to be represented discretely, would never get the repeated visits it asks for.

This paper takes the position between those extremes. We keep ordinary tabular Q-Learning and change what the table is indexed by. Rather than describing the *state*, we describe each candidate *action*: a cell is characterised by what each of its four orthogonal directions runs into first — the board edge, a miss, a run of unexplored water, or a run of hits belonging to a ship that is still afloat — together with the lattice residue of the cell and the size of the smallest enemy ship still afloat. The four direction codes are then canonicalised under the eight-element symmetry group of the square, so that "two hits to my left" and "two hits above me" share one entry. Selecting a shot is an $\arg\max$ over the descriptions of the currently legal cells, and the update is the textbook one. The result is a table with fewer than three thousand reachable entries that trains in under a minute on one CPU core with no dependencies beyond the Python standard library.

Our contributions are:

- **An action-attached local abstraction with symmetry canonicalisation** that reduces Battleship target selection to a tabular problem of a few thousand entries, with the canonicalisation stated as an approximate MDP homo-

morphism and measured (it collapses 13,825 raw keys into 2,839).

- **A reproducible, dependency-free engine, agent API and experiment harness** in which every agent — learned or hand-written — sees exactly the same observation, every run is seeded, and illegal moves are recorded rather than crashing a batch.
- **An empirical comparison** in which the learned agent clears a board in 52.1 shots against 56.8 for a tuned hunt/target baseline and 94.9 for random play, and wins 181 of 300 head-to-head games against the heuristic, with non-overlapping confidence intervals and zero forfeits.
- **An ablation over every design decision**, which shows that canonicalisation (+5.5 shots when removed), run-length resolution (+3.7), optimistic initialisation (+3.2) and the sink bonus (+3.3) all matter — and that one feature the design took for granted, the size of the smallest surviving ship, is actively harmful. Deleting it leaves a 919-entry table that clears boards in 48.2 shots and beats the heuristic 205–95. The binding constraint is the granularity of the key, not the capacity of the table.

Related Work

Battleship as search. The folklore progression — random, hunt/target, hunt/target with a parity sweep, and finally a probability-density agent that scores each cell by how many legal fleet placements cover it — is set out with measurements by Berry (2011), who reports roughly 97 median moves for random play, 66 for hunt/target and 42–44 for the density agent. The density agent is the acknowledged non-learned ceiling and is best read as the Bayesian search-allocation posterior computed exactly (Stone 1975), approximated by sampling (Rochford 2021), or, in a game small enough to solve, by minimax over the consistent hypothesis set (Knuth 1977). Exact fleet-consistency reasoning is not cheap: the puzzle variant is NP-complete (Sevenster 2004), which is precisely why players use heuristics or sampling. Crombez, da Fonseca, and Gerard (2021) give the one rigorous algorithmic treatment of the shooting problem — how to sink a ship of known shape after a first hit while minimising missed shots — and their analysis is the worst-case counterpart of the local "extend the line" rule our agent learns.

Battleship as a learning and planning domain. POMCP (Silver and Veness 2010) and DVRL (Igl et al. 2018) treat Battleship as a $\sim 10^{18}$-state POMDP and attack it with online Monte-Carlo planning and learned belief inference respectively. Clementis (2013) compares supervised and reinforcement adaptation of a neural Battleship player, and Clementis (2014) asks precisely our question — whether the network should be fed the global board or a local description of the neighbourhood — making it the closest prior work. Our answer differs in kind rather than degree: we do not choose an input encoding for a network, we choose an abstraction fine enough to act on and coarse enough to *tabulate*, so that the discreteness proviso of the convergence result (Watkins and Dayan 1992; Tsitsiklis 1994) is satisfied and the resulting policy stays readable. Cauwet and Teytaud (2018) is a useful contrast on cost: benchmarking black-box strategy optimisation over partially observable games, they could afford only about 54 Battleship games within their compute budget, whereas our whole training run is 12,000 games in 55 seconds on one core.

Abstraction, symmetry and local-feature tables. State aggregation has a mature theory: Li, Walsh, and Littman (2006) classify what each family of abstraction preserves, and Abel, Hershkowitz, and Littman (2016) bound the value loss when the abstraction is only approximately value-preserving. Our key is an a^*/Q^*-irrelevance abstraction in that taxonomy — cells with the same local description are assumed to have the same value ordering — and it is only approximately so, which is the honest way to describe what it costs. The canonicalisation step is an approximate MDP homomorphism (Ravindran and Barto 2002) induced by the dihedral group D_4 acting on the neighbourhood, the same device that Zinkevich and Balch (2001) states for symmetric MDPs and that modern deep RL buys with equivariant architectures (van der Pol et al. 2020); in a table it is free, because it is just a normalisation of the key. Methodologically the closest relatives are n-tuple systems, which learn many small lookup tables indexed by local board patterns, for Othello (Lucas 2008) and 2048 (Szubert and Jaśkowski 2014; Jaśkowski 2018), the latter also exploiting board symmetries. Our key is a single, hand-designed, symmetry-canonicalised n-tuple-style feature, and 2,839 entries is three to four orders of magnitude smaller than the n-tuple networks used in 2048. Tile coding (Sutton 1996) is the classical alternative that generalises by overlapping features rather than by aggregation.

Positioning. The field's answer to an unenumerable state space has been to replace the table (Tesauro 1995; Mnih et al. 2015; Silver et al. 2018). We instead shrink the problem until the table is the right tool. Exploration is then handled by the tabular shadow of optimism under uncertainty (Kaelbling, Littman, and Moore 1996; Brafman and Tennenholtz 2002; Machado, Srinivasan, and Bowling 2014), the learning rate by a polynomial schedule with the convergence properties analysed by Even-Dar and Mansour (2003), and the reporting by the standards that Henderson et al. (2018) argues reinforcement learning results need: seeded runs, confidence intervals, and a released harness.

The Game

We state the rules exactly as implemented, since shot counts are not comparable across rule sets. Two players each own a 10×10 board and place the standard fleet — Carrier (5), Battleship (4), Cruiser (3), Submarine (3), Destroyer (2), seventeen of the hundred cells. Ships are axis-aligned and may not overlap; by default they *may* touch, the stricter no-touching house rule being a configuration flag. This matters when comparing with Silver and Veness (2010), whose Battleship POMDP forbids adjacent placement and is therefore a different (easier to deduce) distribution. Players alternate single shots. A hit does *not* grant an extra turn, so the game is a pure race and the first move is worth something; we cancel

that by alternating who starts from game to game. Sinking a ship is announced and, by default, its cells are revealed to the attacker. Firing twice at the same cell or off the board is an illegal move that forfeits the game, recorded in the statistics rather than crashing a batch run.

Three reference points frame the results. Uniform random guessing needs about 95 shots to clear a board and hits 17% of the time. The theoretical floor is 17 shots. And a lattice sweep with spacing k — shooting only cells with $(\text{row} + \text{col}) \bmod k = r$ — cannot miss a ship of length k, which is the observation the parity heuristic exploits and, as we will show, one the learned agent partially rediscovers. Because the smallest ship in the standard fleet has length two, the relevant lattice for most of a game is the checkerboard.

It is worth being precise about how large the problem actually is, because the usual quotation — 4^{100} tracking grids — overstates it. What a player is uncertain about is not the grid but the layout, so the belief state (Kaelbling, Littman, and Cassandra 1998) is a distribution over legal fleet placements. The five ships have 120, 140, 160, 160 and 180 in-bounds placements respectively; a Monte-Carlo estimate over $400{,}000$ independent draws puts the fraction that do not overlap at 0.3882 ± 0.0015, so there are about 3.0×10^{10} legal layouts with touching allowed. That is small enough for sampling methods to bite (Rochford 2021) and far too large to enumerate at every move, which is exactly the gap that heuristics and, here, a learned table are asked to fill.

Finally, we make heavy use of a *solo* variant: one agent alone against a randomly generated board, scored by shots needed to clear it. Formally this is the underlying POMDP (Kaelbling, Littman, and Cassandra 1998) whose belief state is a distribution over legal fleet placements, with the opponent removed. It is the more sensitive measurement, because it deletes first-move luck and the opponent's board draw and leaves only target-selection quality.

Approach

Architecture and the Agent API

The implementation separates three layers that do not leak into one another: the engine (rules, hidden state, turn order, shot resolution), the agents, and the experiment harness. An agent's only view of the opponent is an `Observation` object, which exposes the tracking grid, the legal action set, the hits whose ship is still afloat, and which enemy ships have been sunk. The point is methodological rather than architectural: the hand-written baseline and the learned agent receive exactly the same information through exactly the same interface, so a comparison between them cannot be confounded by one of them peeking. Agent objects are constructed once per experiment and reset per game, so anything learned survives across games; the engine seeds every agent's random generator, so a run with a fixed seed reproduces exactly.

Baselines

RandomAgent shoots uniformly among unexplored cells.

HeuristicAgent is the classic hunt/target strategy, tuned to be the strongest hand-written opponent we could write without placement counting. In *target* mode it groups open hits into orthogonally connected clusters; a cluster of two or more has revealed its axis, so the agent shoots at the two ends of that line, and line extensions always outrank the four neighbours of a lone hit. In *hunt* mode it screens out cells whose unexplored run is too short to hold the smallest enemy ship still afloat, and then restricts itself to the residue class of the lattice $(\text{row} + \text{col}) \bmod k$ that still has the most candidates, where k is that smallest ship. A flag disables the lattice restriction so that we can measure what it is worth.

Code	What the direction runs into first
0	Blocked: board edge, a miss, or a sunk ship
1–4	That many unexplored cells in a row (4 = 4+)
5–7	That many hits of a floating ship (7 = 3+)

Table 1: The direction code, computed once per orthogonal direction of a candidate cell. The key is four such codes, canonicalised, plus the lattice residue and the smallest surviving enemy ship.

Q-Learning over an Action-Attached Abstraction

The learner is textbook tabular Q-Learning,

$$Q(k) \leftarrow Q(k) + \alpha\big(r + \gamma \max_{k'} Q(k') - Q(k)\big), \quad (1)$$

with one difference: the index k is not a state but a *description of an action in its context*. Every candidate cell is mapped to a short tuple, the table holds one value per tuple — "how good is shooting at a cell that looks like this?" — and choosing a shot is an $\arg\max$ over the tuples of the cells that are still legal. The abstraction only decides which raw state-action pairs share an entry; everything else is unchanged.

The key. For each of the four orthogonal directions we record what that direction runs into first, compressed into one small integer (Table 1). Two global components are appended: the lattice residue $(\text{row} + \text{col}) \bmod 2$, which is what allows parity hunting to be discovered rather than programmed, and the size of the smallest enemy ship still afloat, which is what should decide whether a narrow gap is worth a shot. The key is thus six small integers.

Symmetry canonicalisation. Battleship is invariant under the eight symmetries of the square, so "two hits to my left" and "two hits below me" are the same situation and should not be learned twice. We canonicalise by sorting the two codes within each axis and then sorting the two axes against each other, which maps every orbit of D_4 to one representative. This is an approximate MDP homomorphism (Ravindran and Barto 2002): exact for the local description, approximate with respect to the full game, because two cells with the same canonical neighbourhood can still differ in ways the key does not see. The reduction is large but not the full factor of eight — many descriptions are fixed points of some of the symmetries — and we measure it directly: the same training run reaches $13{,}825$ raw keys and $2{,}839$ canonical ones, a collapse of $4.9\times$, with a corresponding multiplication of the samples per entry.

```
  0123456789
0 ....XX....
1 ..#.○.....
2 ..#○....○.
3 ..#..○.#..
4 .○#...○#..
5 ..○○...#○.
6 ....○..#..
7 .○....○#..
8 .....○....
9 ..........
```

Cell	Key	Q
$(0,6)$	$(6,3,3,0),1$	5.71
$(0,3)$	$(6,3,1,0),0$	5.58
$(1,5)$	$(5,1,4,0),0$	4.79
$(9,4)$	$(4,4,2,0),1$	3.94
	71 further cells	
$(3,8)$	$(1,0,1,0),1$	1.45
$(1,3)$	$(1,0,0,0),0$	1.45

Figure 1: The abstraction at work after 24 shots (. unexplored, ○ miss, X hit of a floating ship, # sunk). Each of the 76 legal cells is reduced to a key — four canonical direction codes and the lattice residue — and looked up. The two ends of the wounded ship come first, the perpendicular neighbour next, then open water, and one-cell gaps last: no rule for lines, gaps or parity appears anywhere in the agent.

How small the key space is. With eight possible direction codes the raw tuple space is $8^4 = 4096$; canonicalisation maps it onto 666 orbit representatives, a factor of 6.1. Multiplying by the residue bit and the four possible values of the smallest surviving ship bounds the key space at 32,768 raw and 5,328 canonical entries. What play actually reaches is smaller still, and the measured collapse (13,825 → 2,839, a factor of 4.9) is below the group order of eight because many local descriptions are fixed points of some of the symmetries — a cell surrounded by water is its own mirror image. For comparison, the n-tuple networks used for 2048 (Szubert and Jaśkowski 2014; Jaśkowski 2018) carry millions of weights.

Rewards and schedules. A hit earns $+1$, sinking a ship earns a further $+0.5$, a miss earns 0, and the discount is $\gamma = 0.9$. Discounting is what turns "hit often" into "hit early", which is exactly the shots-to-clear objective. The learning rate decays per entry with its own visit count, $\alpha_t = \max(0.02,\ 0.5/t^{0.55})$; the exponent lies in the range $(1/2, 1)$ that Even-Dar and Mansour (2003) show converges polynomially faster than the $1/t$ schedule while still satisfying the Robbins–Monro conditions. Epsilon anneals linearly from 0.35 to 0.02 over training. Unvisited entries are initialised optimistically at $Q_0 = 6.0$, well above the value a converged entry settles at, so that an unfamiliar description is tried rather than avoided; this is the model-free tabular shadow of R-MAX (Brafman and Tennenholtz 2002; Machado, Srinivasan, and Bowling 2014), and the ablation below shows it is worth 3.2 shots per board. Training runs on solo boards, after which the table is frozen and evaluated greedily.

What the Table Learns

Because the policy is a table over readable descriptions, it can be audited. Table 2 lists the visit-weighted mean value of the entries matching a number of recognisable Battleship situations, taken from the 919-entry table of the best agent we report (the coarse-key variant of the section on ablations; the full-key table gives the same ordering).

Situation	Codes	$\bar{Q}$	Upd.
A hit on each axis	$(5,\cdot,5,\cdot)$	5.83	2,087
Lone hit, water behind	$(5,4,\cdot,\cdot)$	5.54	37,878
Line of 3+ hits	$(7,\cdot,\cdot,\cdot)$	5.38	46,214
Line of 2 hits	$(6,\cdot,\cdot,\cdot)$	5.04	59,663
Gap between two hits	$(5,5,\cdot,\cdot)$	4.62	239
Open water	$(4,4,4,4)$	4.42	4,758
Lone hit, dead end behind	$(5,0,\cdot,\cdot)$	2.51	16,332
Unexplored run of two	$(2,0,0,0)$	1.68	11,240
One-cell gap in misses	$(1,0,0,0)$	1.45	41,965
Boxed in	$(0,0,0,0)$	1.44	42,129

Table 2: Reading the policy out of the table: visit-weighted mean value of all well-visited entries matching each pattern ($\cdot$ is a wildcard). The ordering is a list of Battleship rules nobody wrote down.

The ordering is exactly the strategy a good player would state, and it was never programmed. Cells adjacent to a wounded ship dominate open water; open water dominates cells that cannot hold a ship; and cells boxed in by misses sit at the bottom. Two entries are worth dwelling on. First, "beside a lone hit with a dead end behind" (2.51) is valued far below "beside a lone hit with open water behind" (5.54), and in fact below open water itself: the agent learned that poking at a wound from a direction in which the ship cannot possibly extend is worse than searching fresh territory — a refinement the hand-written baseline does not make. Second, the near-tie at the bottom between a one-cell gap (1.45) and a fully blocked cell (1.44) is the smallest-ship constraint rediscovered: a gap that cannot hold the destroyer is worth no more than a wall.

The lattice residue behaves differently from the direction codes. Comparing entries that differ *only* in the residue bit, the mean absolute difference is just 0.07 and neither class is systematically preferred (the weighted mean difference is -0.01). Yet the residue is not idle: in hunt mode many candidate cells share the same direction codes, so the residue is what breaks the tie, and it breaks it consistently within each context. Behaviourally, over 500 boards the learned agent places 59.2% of its hunting shots on one residue class, against 50.0% for random play, and removing the feature costs 1.4 shots per board. Parity is learned, but as a systematic tie-break rather than as a global preference.

Experimental Setup and Results

Setup

All experiments use the standard 10×10 board and fleet with touching allowed and sunk ships revealed. The Q-table is trained on 12,000 solo boards (seed 20240607), then frozen and evaluated greedily. The headline evaluation is a round robin between `random`, `heuristic` and `qlearning` at 300 games per pair with the first shot alternating between games, plus 300 solo boards per agent, all under seed 0. We report win rate, hit rate, shots to win and solo shots to clear, each with a median and a 95% confidence interval, and we

Agent	Mean	±95%	Median	Max	Hit%
qlearning	52.11	1.17	53	77	32.6
heuristic	56.75	1.35	57	85	30.0
random	94.91	0.60	97	100	17.9

Table 3: Shots needed to clear one board, 300 boards per agent; lower is better. The theoretical floor is 17.

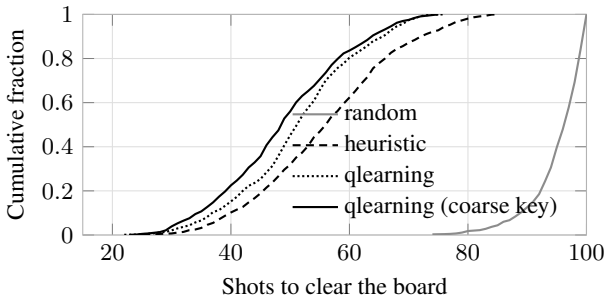


Figure 2: Empirical distribution of shots needed to clear a board, 1,000 boards per agent, the same boards for every agent. The learned curves are both shifted left and steeper than the heuristic's.

track forfeits, which are zero in every run reported here. The engine is pure standard library and every source of randomness is seeded, so the numbers reproduce exactly. Training takes 55 seconds and evaluating a board takes 4 ms on one core of an Intel Core Ultra 9 285K; there is no GPU and no dependency to install.

Solo Board Clearing

Table 3 is the primary comparison. The learned agent saves 4.6 shots per board over the hand-written heuristic and 42.8 over random play, and the confidence intervals are far from overlapping. The shape of the distribution matters as much as its mean: the learned agent's standard deviation is 10.3 against the heuristic's 11.9, and its worst board over 300 needed 77 shots against the heuristic's 85. It is not merely faster on average, it is less exposed to unlucky boards — visible in Figure 2 as a curve that is shifted left and steeper at the same time.

Head-to-Head Duels

Both informed agents beat random play 300–0; with 17 hits needed out of 100 cells, a random player essentially never finishes first. The interesting matchup is `qlearning` against `heuristic`, which the learned agent wins 181–119, a 60.3% win rate (Table 4).

It is worth being precise about why a 4.6-shot advantage becomes a 10-point win-rate advantage, because it also validates the claim that the opponent is passive. If the opponent truly cannot affect our search, then the duel is two independent draws from the two solo distributions, and the player with the smaller draw wins — with ties going to whoever moved first. Computing that prediction from the measured solo distributions gives 61.5% for `qlearning` over `heuristic` under alternating first move, against 60.3% observed. The same calculation predicts 66.7% for the coarse-key agent over the heuristic (observed 68.3%), 56.5% over the full-key agent (observed 57.3%) and 99.9% for the heuristic over random (observed 100%). Duel outcomes in this game are a deterministic consequence of the shots-to-clear distributions, which is exactly why we treat the solo metric as primary.

Agent	Games	Wins	Win%	Hit%	Sh./win
qlearning	600	481	80.2	33.3	50.51
heuristic	600	419	69.8	30.1	54.77
random	600	0	0.0	17.0	—

Table 4: Round robin, 300 games per pair, first move alternating. Zero forfeits. The decisive matchup is `qlearning` vs `heuristic` at 181–119.

	Hunting		Targeting	
Agent	Shots	Hit%	Shots	Hit%
ql-coarse	27.6	16.0	21.5	58.5
qlearning	27.4	16.0	23.6	53.3
heuristic	38.4	11.9	18.6	66.9
random	5.7	17.9	89.4	17.9

Table 5: Shots per board split by mode — *hunting* when no wounded ship is visible, *targeting* otherwise — over 500 solo boards. The learned agents are markedly better hunters and slightly worse finishers.

Where the Advantage Comes From

The aggregate numbers say the learned agent is better; Table 5 says where. Splitting every shot by whether a wounded ship was visible when it was fired separates search from clean-up, and the two learned agents win the first and lose the second. While hunting they need 27.5 shots against the heuristic's 38.4 and hit 16.0% of the time against its 11.9%, a 34% relative improvement in search efficiency, which is where the whole advantage lives. While targeting they are worse: the heuristic converts a wounded ship into a sunk one at 66.9% accuracy, the full-key agent at 53.3%. This is the price of the abstraction. The heuristic reasons about a *cluster* of hits and knows the ship's axis once two are aligned; the learned agent sees only what each individual cell's four directions run into, so a hit run that is interrupted by the board edge or by a second ship lying alongside can leave it without the information the baseline computes explicitly. The decomposition also localises the coarse key's advantage over the full key: the two hunt identically (27.6 and 27.4 shots at the same 16.0%), and the whole difference is in targeting, 21.5 shots at 58.5% against 23.6 at 53.3%. The fragmented table was worst exactly where its samples were thinnest.

Learning Behaviour

Figure 3 shows mean shots per board over the 12,000 training games in blocks of 1,200: 76.1 → 76.0 → 75.3 →

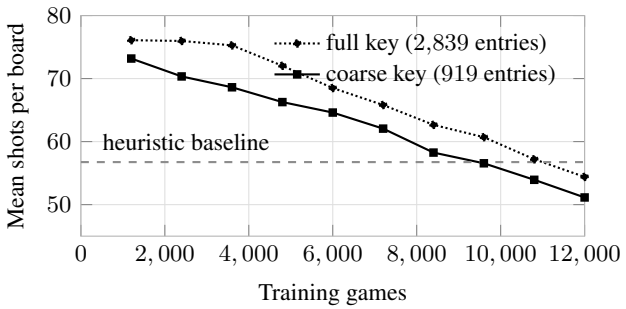


Figure 3: Training curves in blocks of 1,200 games, measured *with* exploration still switched on. The flat opening is optimistic initialisation forcing a systematic sweep through unseen descriptions. Frozen and greedy, the two final tables evaluate at 51.6 and 49.3 shots.

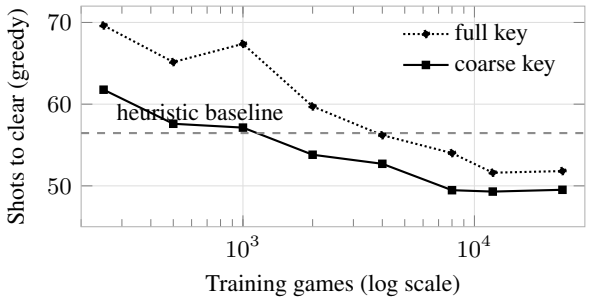


Figure 4: Greedy performance against training budget, 1,000 evaluation boards per point. The coarse key draws level with the heuristic between 1,000 and 2,000 training games and is essentially converged by 8,000; the full key needs about 4,000 to draw level and plateaus higher.

Condition	Entries	Shots	Δ
Full agent	2,839	51.61	—
No canonicalisation	13,825	57.15	$+5.54 \pm 1.08$
No run lengths	140	55.32	$+3.72 \pm 0.90$
No sink bonus	2,902	54.88	$+3.27 \pm 0.94$
No optimistic init.	2,297	54.82	$+3.21 \pm 0.93$
No lattice residue	1,529	53.03	$+1.42 \pm 0.93$
No smallest ship	919	49.30	-2.30 ± 0.81
Direction codes only	475	50.09	-1.51 ± 0.97
Heuristic (parity on)	—	56.46	$+4.85 \pm 0.96$
Heuristic (parity off)	—	56.41	$+4.80 \pm 0.95$
Random	—	95.48	$+43.88 \pm 0.70$

Table 6: Each design decision removed in isolation. Every condition is trained from scratch for 12,000 games and evaluated greedily on the same 1,000 boards, so Δ (shots per board relative to the full agent, positive is worse) is a paired difference with a paired 95% interval.

$72.0 \to 68.5 \to 65.8 \to 62.7 \to 60.7 \to 57.2 \to 54.4$. Two features deserve comment. The first two blocks are nearly flat because optimistic initialisation is doing its job: every unseen description is worth 6.0, so the greedy action is whatever has not been tried, and the agent spends its first couple of thousand games sweeping the key space rather than exploiting it. And the tail is measured with ϵ still nonzero and the table still moving; frozen and evaluated greedily the same table clears in 51.6 shots. The final table holds 2,839 entries after 802,371 updates; 738 of them absorb 90% of those updates, the median entry has been visited 33 times, and at evaluation time only 0.04% of the descriptions the agent encounters have never been seen in training.

Ablations

Table 6 removes one decision at a time. Four results confirm the design. Canonicalisation is the single most valuable piece: without it the key space grows to 13,825 entries, each entry sees roughly a fifth of the samples, and the agent loses 5.5 shots per board. Run-length resolution in the direction codes is worth 3.7 shots: collapsing every code to a bare type leaves only 140 entries and destroys the agent's ability to tell a gap that can still hold a ship from one that cannot. The sink bonus is worth 3.3 shots, since without it the agent has no reason to prefer finishing a wounded ship over wounding another one. Optimistic initialisation is worth 3.2 shots, confirming that directed exploration, not the learning rule, is what the flat opening of the training curve buys; it also leaves the zero-initialised agent with 542 fewer entries, because descriptions it never happens to try never enter the table at all.

Two results contradict expectations. First, the parity lattice is worth much less than folklore suggests — 1.4 shots as a learned residue feature, and *nothing at all* to the hand-written baseline, which scores 56.46 with the sweep on and 56.41 with it off, a difference well inside noise. The explanation is that the baseline already refuses to shoot at cells whose unexplored run is too short for the smallest surviving ship; once that screen is in place, the parity sweep is largely redundant with it. What matters is not the lattice but the fit constraint — which is also why run lengths are the most valuable part of the key.

Second, and more interestingly, the size of the smallest surviving enemy ship is *harmful*. Deleting it improves the agent by 2.3 shots per board and shrinks the table from 2,839 to 919 entries. The feature is genuinely informative — it is what should tell the agent whether a two-cell gap is worth a shot — but the run-length codes already carry most of that information, and conditioning on it splits every description into four weakly-coupled copies that must each be learned separately. Deleting it is therefore not a loss of information so much as a gain in statistical strength: median visits per entry rise from 33 to 336, and the fraction of entries with fewer than five visits falls from 23% to 9%. Dropping the residue as well ("direction codes only", 475 entries) gives back 0.8 shots, so the residue does pay for itself once the table is no longer fragmented.

Figure 4 sweeps the training budget and shows the same story as a sample-efficiency effect. The coarse key overtakes the heuristic before 2,000 training games and flattens by 8,000; the full key needs 4,000 to draw level and is still

Agent	Solo	Win%	Sh./win	Hit%
`ql-coarse`	48.21	75.2	46.60	35.1
`qlearning`	52.02	66.1	49.90	33.5
`heuristic`	56.69	58.7	52.98	30.2
`random`	95.07	0.0	—	17.1

Table 7: Four-agent round robin, 300 games per pair and 300 solo boards per agent. Pairwise: coarse beats the heuristic 205–95 and the full-key agent 172–128; the full-key agent beats the heuristic 167–133.

improving at 12,000, landing higher. Neither curve is an artefact of the budget: at 48,000 training games the full key reaches 51.06 and the coarse key 49.11, so the gap persists at four times the training data. Across five training seeds the full key spans 51.6–53.5 shots and the coarse key 48.5–49.8: the spread due to the training seed is 1.9 shots for one design and 1.3 for the other, and the two ranges do not overlap, so the effect is not a lucky table.

The Coarse-Key Agent

Because the ablation result is a better agent and not merely a diagnostic, we evaluated it under the full protocol (Table 7). A 919-entry table — small enough to print — clears a board in 48.21 ± 1.11 shots, beats the hand-written heuristic 205–95 (68.3%) and beats the full-key learned agent 172–128 (57.3%), with a 35.1% hit rate against the heuristic's 30.2%. It also trains faster, reaching its plateau in about 8,000 games. In this four-agent round robin the full-key agent beats the heuristic 167–133 rather than the 181–119 of Table 4; the two runs use the same protocol but different random streams, and the difference (55.7% against 60.3%) is about 1.2 standard errors — a useful reminder of how wide 300-game duel estimates are, and part of why we report the paired solo metric as primary.

Threats to Validity

We report one board configuration and one fleet, with touching allowed; whether the same key transfers to the no-touching rule, other fleets, or non-square boards is untested, and the no-touching rule in particular changes the placement distribution the agent has implicitly learned. Ablation deltas are paired over 1,000 boards but each condition is a single training run, so a small part of each delta is training noise — we bound it by the five-seed spread above, which is smaller than the effects we claim but not negligible for the 1.4-shot residue result. The abstraction is only approximately value-preserving, and Abel, Hershkowitz, and Littman (2016) is the right lens for what that can cost: two cells with the same canonical description can genuinely differ, and the policy cannot tell them apart. Our baseline is a tuned hunt/target agent, not the probability-density agent, which Berry (2011) reports at 42–44 mean shots on comparable rules; our 48.2 narrows that gap but does not close it, and we do not claim state of the art. Finally, evaluation is against uniformly random fleet placement, not an adversarial placer, and every agent here places its own fleet at random.

Conclusion

A table of 919 entries over a symmetry-canonicalised local description of each cell, trained by ordinary Q-Learning on 12,000 self-play boards with no function approximation, no dependencies and under a minute of single-core compute, clears a Battleship board in 48.2 shots and beats a tuned hunt/target baseline 205–95; the larger 2,839-entry variant we started from clears in 52.1 and wins 181–119. The lesson is not that tabular methods are underrated in general but something more specific: in Battleship the difficulty was never the size of the state space, it was the choice of what to condition on. Once the description is attached to the action and quotiented by the board's symmetries, the problem moves back into the range where a table is not merely sufficient but preferable — the ablations are cheap enough to run exhaustively, and the resulting policy can be read out and checked line by line, which is how we discovered that the agent had learned not to poke a wounded ship from a direction it cannot extend into.

The same experiments say something a little uncomfortable about feature engineering. The one component of the key that was included on the strength of a strategic argument rather than a measurement — the size of the smallest surviving ship — is the one that turned out to hurt, because its cost in statistical strength exceeded its value in information. With the table small enough to ablate exhaustively, that was a two-minute experiment to run; with a network in place of the table it would have been a much easier mistake to keep.

Future Work

The obvious next step is a probability-density opponent — scoring each cell by how many legal fleet placements cover it — both as the upper-bound baseline the field acknowledges and as a source of features: a coarse bucket of the placement count would bring the key closer to the Bayesian posterior while keeping it tabular. Beyond that, the ablation result invites a systematic search over key granularity rather than the hand-designed keys we tried, and a linear or small convolutional Q-function over the same features would establish whether the table or the features are the binding constraint. Placement is the other half of the game and is currently uniformly random on both sides; learning a placement policy against a learned shooter would make the self-play genuine. Finally, the harness supports keeping learning switched on during play, which opens opponent modelling and within-match adaptation, and the engine already exposes the no-touching rule, hidden sunk-ship cells and alternative fleets, so measuring how much of a learned table transfers between rule sets requires no new machinery.

Reproducibility

The engine, the agents, the experiment harness, the ablation scripts and the trained tables are contained in the accompanying repository, which has no dependencies beyond the Python standard library. Every number in this paper is produced by `scripts/run_comparison.py` (Tables 3 and 4), `scripts/run_ablations.py` (Table 6 and Figures 3 and 4), `scripts/run_followups.py`

(Table 7) and `paper/analysis.py` (Tables 2 and 5, Figures 1 and 2), all of which are seeded and write their raw output as JSON. The whole set of experiments reported here, ablations included, is 39 training runs and about six minutes of wall-clock time on a 24-core desktop.

References

Abel, D.; Hershkowitz, D. E.; and Littman, M. L. 2016. Near Optimal Behavior via Approximate State Abstraction. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*.

Berry, N. 2011. Battleship. DataGenetics blog, http://www.datagenetics.com/blog/december32011/.

Brafman, R. I.; and Tennenholtz, M. 2002. R-MAX — A General Polynomial Time Algorithm for Near-Optimal Reinforcement Learning. *Journal of Machine Learning Research*, 3: 213–231.

Brown, N.; and Sandholm, T. 2018. Superhuman AI for Heads-Up No-Limit Poker: Libratus Beats Top Professionals. *Science*, 359(6374): 418–424.

Cauwet, M.-L.; and Teytaud, O. 2018. Surprising Strategies Obtained by Stochastic Optimization in Partially Observable Games. In *IEEE Congress on Evolutionary Computation (CEC)*. ArXiv:1807.01877.

Clementis, L. 2013. Supervised and Reinforcement Learning in Neural Network Based Approach to the Battleship Game Strategy. In *Nostradamus 2013: Prediction, Modeling and Analysis of Complex Systems*, volume 210 of *Advances in Intelligent Systems and Computing*. Springer.

Clementis, L. 2014. Global and Local Environment State Information as Neural Network Input by Solving the Battleship Game. In *Nostradamus 2014: Prediction, Modeling and Analysis of Complex Systems*, Advances in Intelligent Systems and Computing, 291–300. Springer.

Crombez, L.; da Fonseca, G. D.; and Gerard, Y. 2021. Efficient Algorithms for Battleship. In *10th International Conference on Fun with Algorithms (FUN 2021)*, volume 157 of *LIPIcs*, 11:1–11:15. Schloss Dagstuhl–Leibniz-Zentrum für Informatik. ArXiv:2004.07354.

Even-Dar, E.; and Mansour, Y. 2003. Learning Rates for Q-Learning. *Journal of Machine Learning Research*, 5: 1–25.

Grand, G.; Pepe, V.; Andreas, J.; and Tenenbaum, J. B. 2024. Loose LIPS Sink Ships: Asking Questions in Battleship with Language-Informed Program Sampling. In *Proceedings of the Annual Meeting of the Cognitive Science Society (CogSci)*. ArXiv:2402.19471.

Grand, G.; Pepe, V.; Andreas, J.; and Tenenbaum, J. B. 2026. Shoot First, Ask Questions Later? Building Rational Agents that Explore and Act Like People. In *International Conference on Learning Representations (ICLR)*. ArXiv:2510.20886.

Henderson, P.; Islam, R.; Bachman, P.; Pineau, J.; Precup, D.; and Meger, D. 2018. Deep Reinforcement Learning that Matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*. ArXiv:1709.06560.

Igl, M.; Zintgraf, L.; Le, T. A.; Wood, F.; and Whiteson, S. 2018. Deep Variational Reinforcement Learning for POMDPs. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*. ArXiv:1806.02426.

Jaśkowski, W. 2018. Mastering 2048 with Delayed Temporal Coherence Learning, Multi-Stage Weight Promotion, Redundant Encoding and Carousel Shaping. *IEEE Transactions on Games*. ArXiv:1604.05085.

Kaelbling, L. P.; Littman, M. L.; and Cassandra, A. R. 1998. Planning and Acting in Partially Observable Stochastic Domains. *Artificial Intelligence*, 101(1–2): 99–134.

Kaelbling, L. P.; Littman, M. L.; and Moore, A. W. 1996. Reinforcement Learning: A Survey. *Journal of Artificial Intelligence Research*, 4: 237–285.

Knuth, D. E. 1977. The Computer as Master Mind. *Journal of Recreational Mathematics*, 9(1): 1–6.

Li, L.; Walsh, T. J.; and Littman, M. L. 2006. Towards a Unified Theory of State Abstraction for MDPs. In *9th International Symposium on Artificial Intelligence and Mathematics (ISAIM)*.

Lucas, S. M. 2008. Learning to Play Othello with N-Tuple Systems. *Australian Journal of Intelligent Information Processing Systems*, 4: 1–20.

Machado, M. C.; Srinivasan, S.; and Bowling, M. 2014. Domain-Independent Optimistic Initialization for Reinforcement Learning. ArXiv:1410.4604.

Mnih, V.; Kavukcuoglu, K.; Silver, D.; et al. 2015. Human-Level Control Through Deep Reinforcement Learning. *Nature*, 518: 529–533.

Ravindran, B.; and Barto, A. G. 2002. Model Minimization in Hierarchical Reinforcement Learning. In *Abstraction, Reformulation, and Approximation (SARA)*, volume 2371 of *LNCS*. Springer.

Rochford, A. 2021. Playing Battleship with Bayesian Search Theory, Thompson Sampling, and Approximate Bayesian Computation. https://austinrochford.com/posts/2021-09-02-battleship-bayes.html.

Rothe, A.; Lake, B. M.; and Gureckis, T. M. 2017. Question Asking as Program Generation. In *Advances in Neural Information Processing Systems 30 (NIPS)*. ArXiv:1711.06351.

Rothe, A.; Lake, B. M.; and Gureckis, T. M. 2018. Do People Ask Good Questions? *Computational Brain & Behavior*, 1: 69–89.

Sevenster, M. 2004. Battleships as a Decision Problem. *ICGA Journal*, 27(3): 142–147.

Silver, D.; Hubert, T.; Schrittwieser, J.; et al. 2018. A General Reinforcement Learning Algorithm that Masters Chess, Shogi and Go through Self-Play. *Science*, 362(6419): 1140–1144.

Silver, D.; and Veness, J. 2010. Monte-Carlo Planning in Large POMDPs. In *Advances in Neural Information Processing Systems 23 (NIPS)*.

Stone, L. D. 1975. *Theory of Optimal Search*. Academic Press.

Sutton, R. S. 1996. Generalization in Reinforcement Learning: Successful Examples Using Sparse Coarse Coding. In *Advances in Neural Information Processing Systems 8 (NIPS)*, 1038–1044.

Szubert, M.; and Jaśkowski, W. 2014. Temporal Difference Learning of N-Tuple Networks for the Game 2048. In *IEEE Conference on Computational Intelligence and Games (CIG)*.

Tesauro, G. 1995. Temporal Difference Learning and TD-Gammon. *Communications of the ACM*, 38(3): 58–68.

Tsitsiklis, J. N. 1994. Asynchronous Stochastic Approximation and Q-Learning. *Machine Learning*, 16(3): 185–202.

van der Pol, E.; Worrall, D. E.; van Hoof, H.; Oliehoek, F. A.; and Welling, M. 2020. MDP Homomorphic Networks: Group Symmetric Reinforcement Learning. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*.

Watkins, C. J. C. H.; and Dayan, P. 1992. Technical Note: Q-Learning. *Machine Learning*, 8(3–4): 279–292.

Zinkevich, M.; and Balch, T. 2001. Symmetry in Markov Decision Processes and its Implications for Single-Agent and Multi-Agent Learning. In *Proceedings of the 18th International Conference on Machine Learning (ICML)*.

Zinkevich, M.; Johanson, M.; Bowling, M.; and Piccione, C. 2007. Regret Minimization in Games with Incomplete Information. In *Advances in Neural Information Processing Systems 20 (NIPS)*.